POO C#

A2 - Kévin Roussel

Debuggage

- Il est important, fondamental de comprendre et de savoir utiliser les outils de debuggage.
- Beaucoup ne font que des Debug.Log lors d'utilisation de moteurs et c'est une très mauvaise pratique lorsqu'elle est systématique : c'est très pauvre en information.
- Lorsqu'il y a un doute, il est pertinent de placer un breakpoint puis d'avancer pas à pas pour comprendre le comportement de notre travail et les différents aller/retour.

Démo live Visual Studio & Rider

Visibilité

Pour chaque champs, propriétés et méthodes on peut spécifier un niveau d'accessibilité. Ce membre ne sera visible que pour une catégorie précise.

private : Disponible que pour la classe elle-même

public : Disponible auprès de tout le monde.

internal : Disponible qu'au sein de la même DLL (Assembly Definition sur Unity)

protected : Disponible pour la classe elle-même ainsi que pour les classes enfants

Heritage

Héritage

Certains composants ont potentiellements des méthodes en commun, des champs en communs.

Au lieu de copier/coller, utilisons l'héritage

Le but : rassembler du code

L'enfant récupère tout le contenu de la classe mère

+ on peut ajouter de nouvelles méthodes / champs / propriétés / event
+ on peut modifier les méthodes que l'on a récupéré (si elles sont virtuelles)

virtual, override

Une méthode virtuelle peut être réécrite par une classe enfant

Si un enfant veut réécrire le contenu, il faut override

base

On a toujours à disposition la méthode de base fournie par la classe mère.

```
base.Use();
```

Nouvelle visibilité : protected

Rendre disponible un membre aux classes qui héritent de nous

sealed : fermer l’héritage

une classe marquée comme sealed ne peut pas être hérité.

Exemple : Item

public virtual void Use()

un Item dans le jeu représente un objet que le joueur peut attraper

- ça peut être une potion, une fois utilisé on gagne de la vie
- ça peut être une arme, une fois utilisé on est équipé de l'arme
- ça peut être une clé, une fois utilisé on ouvre une porte
- ça peut être une bicyclette, une fois utilisé on peut rouler dans le jeu

abstract

une méthode virtuelle, sans contenu.

Chaque classe enfant doit remplir la méthode afin de pouvoir être valide.

is

mot clé pour savoir si un objet EST tel type

as & cast

Des fois, on a une liste d'Item ...

mais on SAIT que l'item que l'on a sous la main est en fait une potion, on veut accéder à un champs unique à lui.

On peut choisir de caster l'objet, "Cher unity, je sais que cet objet est une potion, promis juré"

Le cast “méchant”

Considère ça comme une potion, et si je me suis trompé : envoi moi une erreur fatale

```
Item monObjet;

Potion p = (Potion) monObjet;
```

Le cast “gentil”

Considère ça comme une potion, et si je me suis trompé met moi juste “null” comme valeur, my bad.

Item monObjet;

Potion p = monObjet as Potion;

L’héritage ajoute :

virtual, override, base

abstract (class & méthode)

protected

sealed

cast

3 types dont vous hériterez souvent :

- MonoBehaviour, le composant Unity

- ScriptableObject, le fichier de configuration

- StateMachineBehaviour, le composant d'Animator

C’est le moment de bosser

...

Résolvons des TU avec de l’héritage

Comme d’habitude voici un jeu de tests unitaire :

à vous de résoudre toutes les situations :

Le package :

https://drive.google.com/file/d/1D06HzYlT9-QjxcMK-zj7sAOJLv5rnFjP/view?usp=sharing